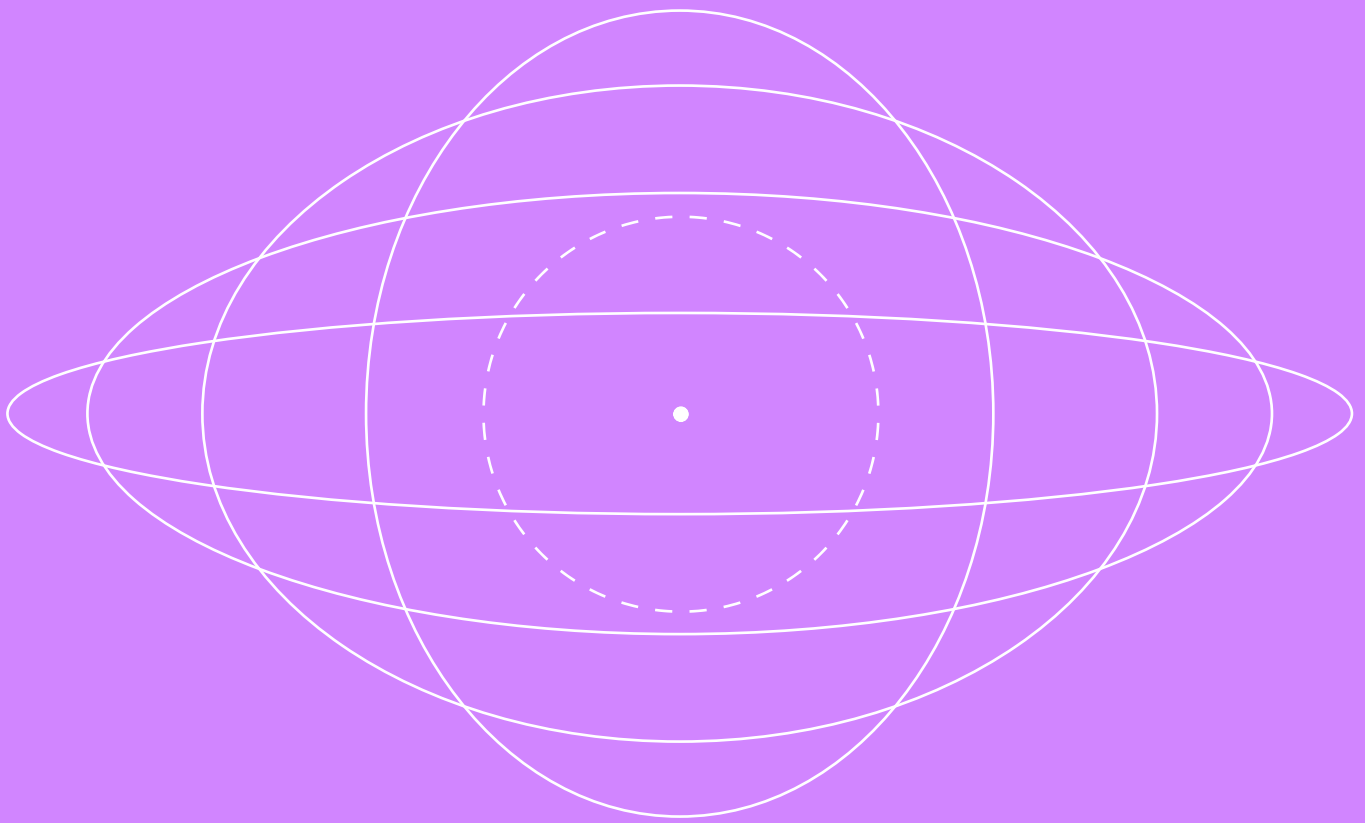


Индексы.

Типы индексов



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

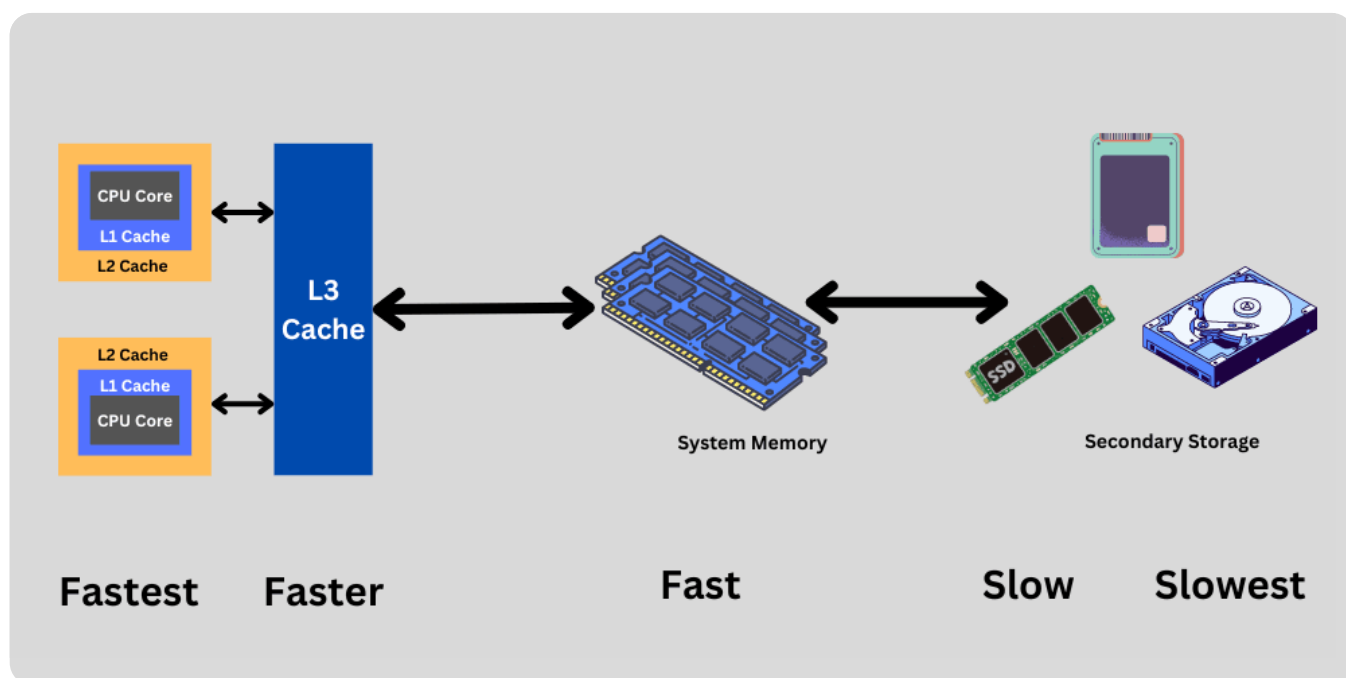
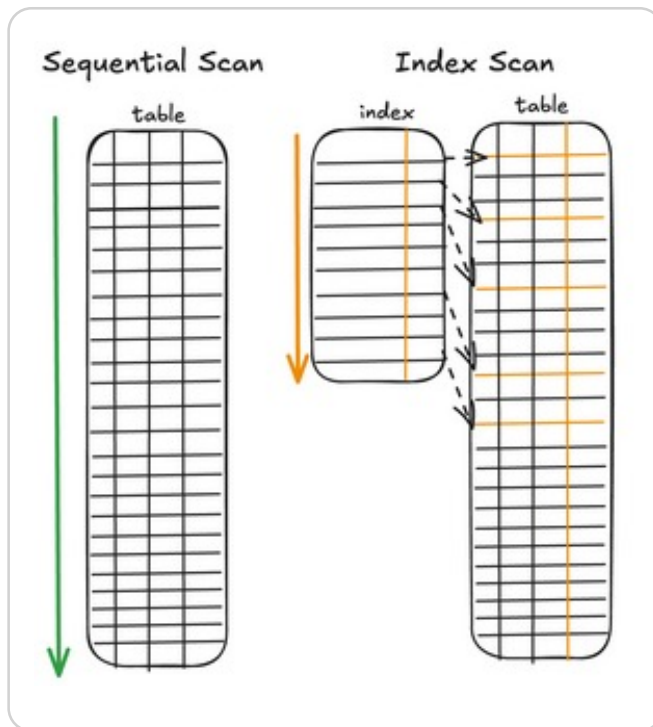
ЛОНГРИД

НЕДЕЛЯ 9

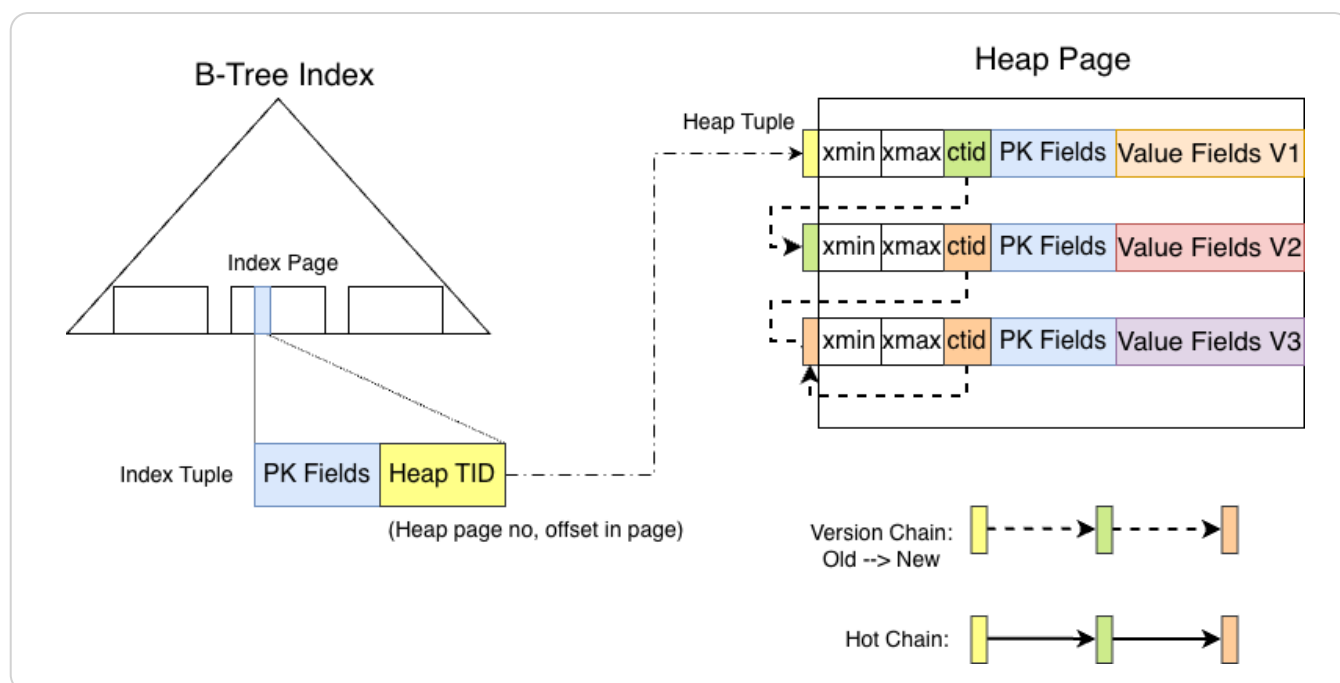
Сегодня мы будем говорить о скорости. О том, как превратить запрос, который выполняется минуты, в запрос, который обрабатывает за миллисекунды. Мы будем говорить об индексах. Это не просто «ещё одна фишка» базы данных. Понимание индексов — это основа производительности твоих приложений, и именно это знание часто отличает начинающего разработчика от опытного инженера.

Наш план на сегодня — это полное погружение. Мы начнём с самой сути проблемы: почему запросы вообще могут быть медленными. Затем разберём, как индексы решают эту проблему. Мы детально изучим самый главный тип индекса — B-Tree. Познакомимся со специализированными инструментами для сложных данных. Научимся «читать мысли» PostgreSQL с помощью команды EXPLAIN. И в конце рассмотрим несколько способов, которые помогут тебе выжать максимум производительности из твоих запросов.

Начнём с основ. Как PostgreSQL ищет данные, если мы ему никак не помогаем? Он использует метод, который называется последовательное сканирование, или Sequential Scan. Представь, что таблица на диске — это огромный стеллаж с книгами. Твой запрос `WHERE title = 'Война и мир'` — это задание найти одну конкретную книгу. PostgreSQL в этом случае ведёт себя как очень дотошный, но не очень умный библиотекарь. Он подходит к самой первой книге на самой первой полке, смотрит на её название, видит, что это не то, ставит обратно. Берёт вторую. Третью. И так далее, книгу за книгой, полку за полкой, пока не дойдёт до самого конца. Даже если он найдёт нужную книгу в середине, он может продолжить сканирование, если не уверен, что она единственная. Главная проблема такого подхода — его производительность линейно деградирует с ростом таблицы. В 10 раз больше книг — в 10 раз дольше поиск.



Почему Seq Scan — это медленно? Главный враг производительности любого приложения, работающего с данными, — это дисковый ввод-вывод (I/O). Чтение данных с жёсткого диска, или даже с SSD — это физический процесс, который в тысячи, а то и в десятки тысяч раз медленнее, чем чтение тех же данных из оперативной памяти (RAM). Когда PostgreSQL выполняет Seq Scan по таблице, которой ещё нет в его кеше (shared_buffers), он вынужден читать с диска страницу за страницей. Seq Scan убивает производительность не только дисковым I/O. Когда страница поднята в shared_buffers, начинается Tuple Deforming — десериализация бинарного представления кортежа в память. Далее CPU выполняет Visibility Check: для каждой строки проверяются xmin/xmax в заголовке MVCC, чтобы понять, видна ли эта версия строки текущей транзакции. На широких таблицах (сотни колонок) этот процесс выжигает CPU до 100% даже при нулевом обращении к диску. Наша главная цель при оптимизации — минимизировать количество обращений к диску.



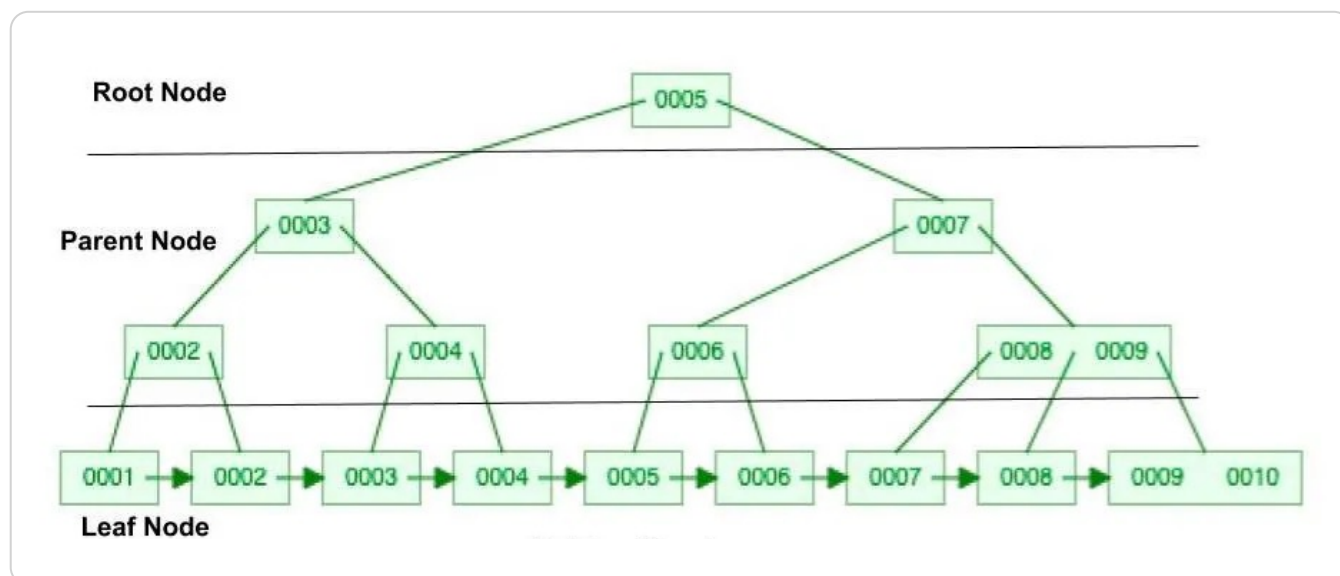
Как решают эту проблему в реальной библиотеке? С помощью картотеки. Индекс в базе данных — это и есть такая картотека. Это отдельная вспомогательная структура данных. В нашей аналогии это щик с карточками, отсортированными по названию книги. На каждой карточке написано название и точный адрес на полке — ряд, шкаф, полка. Теперь наш библиотекарь не бежит к стеллажам. Он подходит к маленькой картотеке, благодаря алфавитному порядку мгновенно находит карточку «Война и мир», видит адрес и идёт напрямую к нужной полке. Вместо того чтобы перебрать миллион книг, он делает всего два действия: быстрый поиск в картотеке и один прыжок к данным. Этот «адрес» в PostgreSQL называется ctid — это физический указатель на версию строки на диске. Фундаментальное отличие PostgreSQL от, например, MySQL (InnoDB) состоит в том, что вторичные индексы ссылаются напрямую на физическое расположение строки (heap), а не на Primary Key. Это ускоряет первичный поиск (один прыжок вместо двух), но порождает критическую уязвимость: при перемещении строки на другую страницу (например, при UPDATE) СУБД обязана обновить указатели ctid абсолютно во ВСЕХ индексах этой таблицы, что вызывает лавинообразный Index Bloat.

Важно понимать, что Index Scan — это почти всегда двухэтапный процесс. **Шаг первый:** PostgreSQL бежит по маленькой и быстрой структуре индекса (которая, скорее всего, уже находится в оперативной памяти) и находит там ctid нужных нам строк. **Шаг второй:** получив этот адрес, он делает «прыжок» в основную, большую таблицу (которую называют кучей или heap) и уже оттуда забирает все остальные поля строки, которые мы запросили (SELECT *). Наша цель при оптимизации — сделать оба этих шага максимально быстрыми, а в идеале — вообще избежать второго шага, о чём мы поговорим позже.

Увы, индексы — это не бесплатно. Это компромисс. Во-первых, картотека — это, по сути, **дублирование информации**, и она занимает место на диске. Иногда индекс может быть даже больше самой таблицы. Во-вторых, и это самое главное, картотеку нужно **поддерживать в актуальном состоянии**. Каждый раз, когда в библиотеку поступает новая книга (INSERT), нужно добавить новую карточку. Когда книгу списывают (DELETE), карточку нужно убрать. А когда книгу переставляют (UPDATE), карточку нужно обновить. Все эти операции замедляют запись данных. Поэтому «золотое правило» — **индексы ускоряют SELECT, но замедляют INSERT, UPDATE и DELETE**. Не создавай индексы «на всякий случай». Каждый индекс должен быть оправдан.

Небольшой пример: поле btpo_next = 2 — это тот самый спасательный трос (указатель на следующую страницу). Если транзакция А прямо сейчас расщепляет страницу 1, а транзакция В ищет ключ, который был перенесён, она не падает с ошибкой консистентности. Транзакция В захватит легковесную блокировку Share Latch, прочтает btpo_next и бесшовно перейдёт на страницу 2.

99% индексов, которые ты будешь создавать, — это **B-Tree**, или сбалансированное дерево. Это структура данных, которая идеально подходит для задач, где важен порядок. Представь её как перевернутую генеалогическую схему. Наверху «корень», от него отходят «ветви», и в самом низу — «листья». В листьях и хранятся наши отсортированные данные и указатели (ctid) на строки в таблице. Главная фишка B-Tree — оно **всегда сбалансировано**. Это значит, что «путь» от корня до любого листа всегда имеет одинаковую длину. Нет «длинных» и «коротких» веток. Это гарантирует предсказуемое и очень быстрое время поиска.



Поиск в B-Tree невероятно эффективен. Допустим, мы ищем число 6. Мы начинаем с корневого узла. Смотрим на ключи в нём. 6 больше 5? Да. Значит, нам нужно идти по правой «ветке». Спускаемся на уровень ниже. 6 меньше 7? Да. Идём по левой «ветке». Мы пришли в листовой узел, где быстро находим 6 и связанный с ним ctid. Всё, поиск завершён. Глубина такого дерева растёт логарифмически. Для таблицы с миллиардом строк B-дереву потребуется всего 4 или, может быть, 5 уровней. Это означает, что для поиска любого значения PostgreSQL нужно сделать всего 4–5 логических чтений вместо миллионов при полном сканировании. Выигрыш — в сотни тысяч раз.

PostgreSQL использует не классическое B-Tree, а модификацию B-Link Tree. Наличие right-links (горизонтальных связей между узлами) решает проблему конкурентного доступа. Когда происходит вставка и страница расщепляется (Page Split), СУБД не накладывает тяжеловесные блокировки на всё дерево. Если параллельный SELECT спускается к листу и обнаруживает, что искомого ключа там больше нет (он уехал вправо при сплите), сканирование просто переходит по right-link на соседнюю страницу, используя лишь кратковременные защёлки памяти (Buffer LWLocks/Latches), а не транзакционные блокировки.

`n_tup_hot_upd = 10000` означает, что мы сэкономили 10 000 операций записи в B-Tree-индекс по `id`. Строка обновлялась внутри той же страницы (heap-only tuple). Если бы FILLFACTOR был равен 100 (по умолчанию), странице не хватило бы места, произошёл бы перенос строки на новую страницу, и индекс пришлось бы перестраивать.

А что происходит с индексом при изменении данных? При DELETE строка в основной таблице помечается как «мёртвая». В самом индексе соответствующая запись сразу не удаляется, а лишь помечается как недействительная. Физически место будет очищено только ввремя VACUUM. А UPDATE — это, как мы помним, DELETE + INSERT. Это значит, что старая запись в индексе помечается как «мёртвая», а новая запись вставляется в нужное место в дереве. Если на листовой странице индекса уже нет места для новой записи, происходит дорогостоящая операция «расщепления страницы», которая может замедлить вставку. PostgreSQL имеет механизм спасения производительности — HOT (Heap-Only Tuples). Если UPDATE не изменяет индексируемые столбцы и на странице есть свободное место, новая версия строки записывается на ту же страницу памяти. Индексы при этом НЕ ОБНОВЛЯЮТСЯ. Старый ctid начинает указывать на цепочку версий внутри страницы. Чтобы этот механизм работал на высоконагруженных таблицах, инженеры снижают параметр FILLFACTOR (например, до 80–90%), резервируя место на страницах специально под будущие HOT-обновления. Запрос `SELECT n_tup_hot_upd FROM pg_stat_user_tables` — твой главный инструмент контроля.

Но есть нюансы.

- **Сценарий:** есть таблица `users` с индексами по `email` и `id`. Система часто обновляет поле `last_seen` (время последнего захода). Работает оптимизация HOT, индексы не трогаются.
- **Действие:** добавляется индекс на `last_seen`. -> Оптимизация HOT отключается для этого поля. Теперь каждый UPDATE `last_seen` принудительно обновляет записи во ВСЕХ ТРЁХ индексах (`id`, `email`, `last_seen`).
- **Результат:** нагрузка на диск (Write IOPS) вырастает кратно, хотя объём полезных данных изменился на байты.

Благодаря своей отсортированной структуре, B-Tree — это универсальный солдат. Он идеален для точечных запросов по равенству, великолепен для диапазонных запросов, может ускорять сортировку (ORDER BY) и поддерживает префиксный поиск, как LIKE 'admin%', потому что все слова, начинающиеся на 'admin', лежат в индексе рядом.

Важный нюанс: в отличие от некоторых других СУБД, PostgreSQL индексирует NULL-значения. Это значит, что NULL хранятся в B-Tree как обычные значения и, следовательно, запрос WHERE column IS NULL может очень эффективно использовать индекс. Это полезно, но также означает, что если у тебя в столбце очень много NULL, они будут занимать место в индексе.

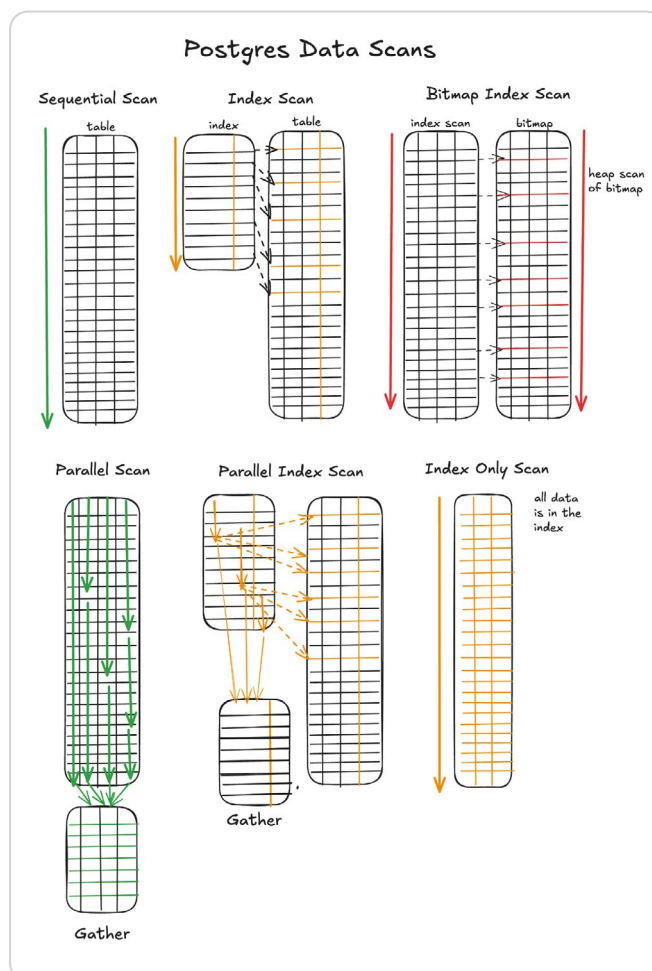
Очень часто мы ищем по нескольким полям. Правильное решение — создать один составной индекс. Представь его как телефонный справочник, который отсортирован сначала по фамилии, а для всех людей с одинаковой фамилией — по имени.

Порядок столбцов в таком индексе критически важен! Индекс по (city, registration_date) будет отлично работать для поиска по городу и ещё лучше для поиска по городу и дате. Но он будет практически бесполезен, если ты попытаешься найти всех пользователей по дате регистрации, не указав город. Ты не можешь использовать вторую часть индекса, не использовав первую. Это как пытаться найти в телефонной книге всех Иванов, не зная их фамилий.

Когда ты отправляешь SQL-запрос, планировщик запросов выступает в роли стратега. Он анализирует твой запрос, смотрит на имеющиеся индексы, на статистику распределения данных в таблицах, и генерирует несколько возможных «планов» выполнения. Затем он оценивает «стоимость» каждого плана (в условных единицах, примерно равных одному чтению страницы с диска) и выбирает самый дешёвый. В 99% случаев он умнее нас.

Чтобы узнать, какой план выбрал планировщик, есть команда EXPLAIN. Ты уже знаешь эту команду, нам здесь самое важное, на что нужно обратить внимание — это тип операции: Index Scan (хорошо) или Seq Scan (повод задуматься).

Читая план, ты встретишь разные способы сканирования. Seq Scan — полное сканирование. Index Scan — поиск по индексу и прыжок в таблицу. Index Only Scan — идеальный сценарий, когда все данные берутся только из индекса. Тут надо отметить, что, даже если все запрашиваемые столбцы есть в листовом узле индекса, Index Only Scan не гарантирует отсутствие чтения из таблицы. В PostgreSQL индексы не хранят информацию о транзакционной видимости (MVCC). Найдя данные в индексе, планировщик обязан обратиться к Visibility Map (VM). Если страница помечена как all-visible, база отдаёт данные. Если бит не выставлен (на странице были недавние изменения), PostgreSQL делает принудительный Heap Fetch — прыжок в основную таблицу только ради чтения



заголовка строки. Если автовакуум настроен плохо, твой Index Only Scan будет генерировать тысячи Heap Fetches, сводя оптимизацию к нулю. Ищи метрику Heap Fetches в выводе EXPLAIN ANALYZE. И, наконец, Bitmap Scan — это хитрый двухэтапный процесс. Сначала по индексу создаётся «карта битов» подходящих строк, а потом по этой карте эффективно вычитываются страницы из таблицы. Он хорош, когда индекс возвращает не одну, а существенную часть строк.

Для JOIN тоже есть несколько стратегий. Nested Loop — это простой перебор, хорош для маленьких таблиц. Hash Join — очень быстрый для соединений по равенству, он строит хеш-таблицу в памяти. Merge Join — самый эффективный, если обе таблицы уже отсортированы по ключу соединения, он их просто «сливает» за один проход.

Посмотрим на реальный пример. У нас есть медленный JOIN. EXPLAIN ANALYZE «ДО» показывает, что PostgreSQL вынужден полностью сканировать обе таблицы. Мы анализируем WHERE и ON и создаём правильные индексы. EXPLAIN ANALYZE «ПОСЛЕ» показывает, что план кардинально изменился: теперь используются Index Scan, и время выполнения сократилось в сотни раз. Это и есть ежедневная работа по оптимизации.

B-Tree — это король, но он не всемогущ. Он отлично работает с простыми данными, которые можно отсортировать: числа, строки, даты. Но что, если нам нужно найти все посты с тегом 'sql' внутри массива тегов? Или найти всех пользователей, у которых в JSON-поле есть ключ status со значением active? B-Tree здесь бессилен, потому что он не умеет «заглядывать» внутрь сложных структур. Для таких задач в PostgreSQL есть целый арсенал специализированных индексов.

Для поиска «внутри» чего-то используется **GIN-индекс**. Его аналогия — алфавитный указатель в конце научной книги. Представь, у тебя есть массив тегов. GIN-индекс «разберёт» этот массив на отдельные теги ('sql', 'postgres', 'performance') и для каждого из них создаст список «страниц» (то есть ctid-строк), где этот тег встречается. Когда ты делаешь запрос «найди все строки с тегом sql», PostgreSQL мгновенно находит 'sql' в своём указателе и получает готовый список адресов нужных строк. Это невероятно эффективно для индексации массивов, JSONB и для полнотекстового поиска.

Стандартный GIN-индекс по JSONB индексирует **каждый ключ и каждое значение** в документе. Это позволяет мгновенно находить документы, где есть определённый ключ (?), или где ключ имеет определённое значение (@>). Это невероятно мощно для работы с полуструктурированными данными.

Другой интересный тип — GiST-индекс. Если GIN хорош для проверки «содержит ли?», то GiST хорош для проверки «пересекается ли?» или «находится ли рядом?». Он используется для данных, где нет чёткого линейного порядка, но есть понятие близости или вложенности.

Самый яркий пример — геоданные. GiST-индекс по геометрии строит R-дерево, которое группирует близкие точки в «ограничивающие прямоугольники». Запрос «найди все в радиусе 5 км» сначала быстро отбрасывает те группы прямоугольников, которые точно не пересекаются с нашим кругом поиска, и работает только с потенциально подходящими объектами.

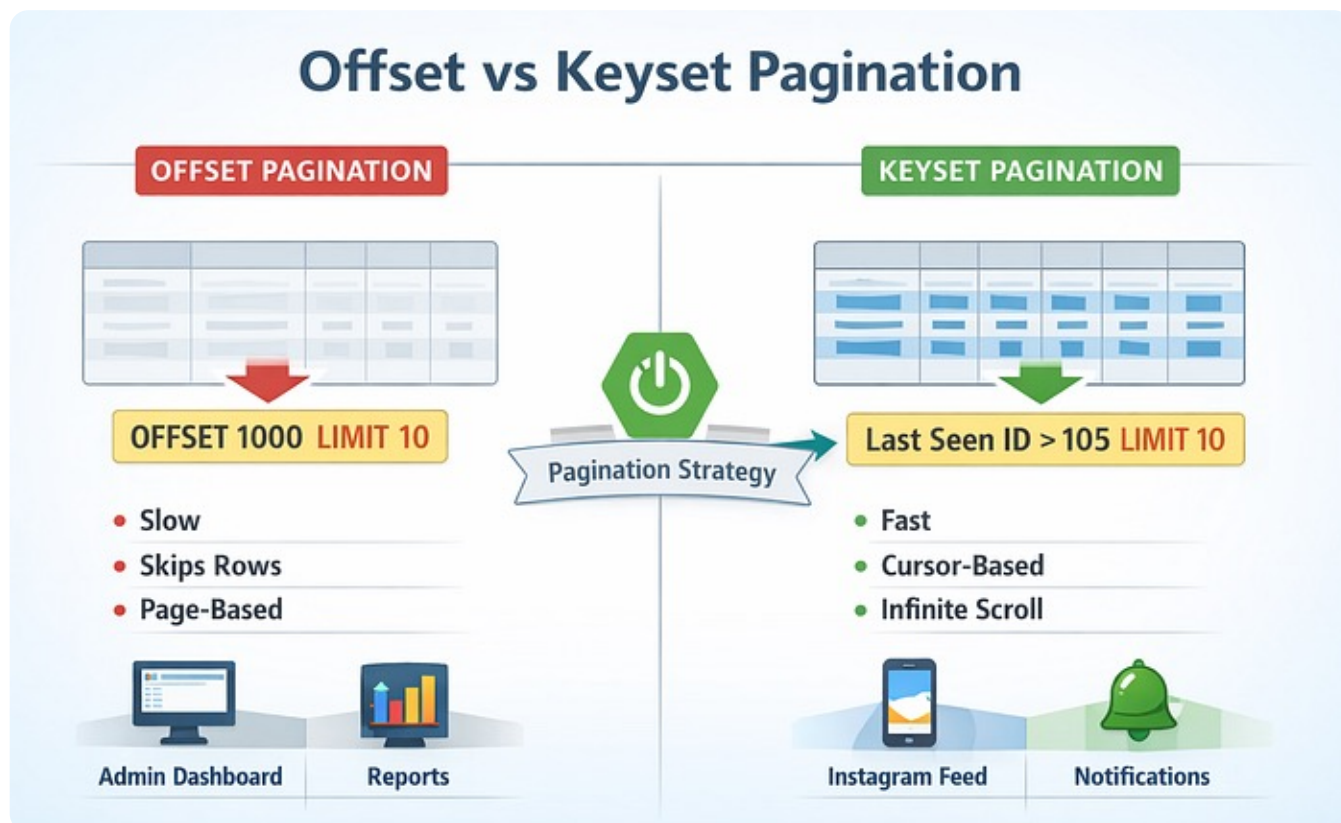
BRIN-индекс — это особый зверь. Он предназначен для гигантских, просто чудовищно больших таблиц, у которых есть одна важная особенность: данные в них физически упорядочены на диске. Классический пример — таблица логов или событий, куда записи постоянно дописываются в конец и, таким образом, отсортированы по времени.

Сравним. На миллиарде строк логов обычный B-Tree-индекс будет занимать десятки гигабайт и сильно замедлять вставку. BRIN-индекс будет занимать всего несколько мегабайт, почти не влиять на вставку и давать очень быстрый, хоть и не мгновенный, отклик на запросы по диапазону дат. Если у тебя гигантская, отсортированная по природе таблица, и тебе не нужна сверхнизкая задержка, BRIN — идеальный выбор.

Поговорим про оптимизацию. Частичный индекс — это индекс, который строится не по всей таблице, а только по её части. Представь таблицу заказов, где 99% заказов имеют статус 'completed', и мы их никогда не ищем. Зато мы постоянно ищем 1% заказов со статусом 'pending'. Зачем нам создавать огромный индекс по всей таблице? Мы можем сказать PostgreSQL: `CREATE INDEX ... WHERE status = 'pending'`. Он создаст крошечный и сверхбыстрый индекс только для тех строк, которые нам действительно интересны.

Ещё одна мощная техника оптимизации. Допустим, нам нужно получить email и created_at для пользователя с определённым user_id. Мы создаём B-Tree-индекс на user_id. PostgreSQL быстро находит по нему нужную строку, но потом ему всё равно нужно сделать «прыжок» в основную таблицу, чтобы оттуда забрать email и created_at. Но мы можем сказать: `CREATE INDEX ON users (user_id) INCLUDE (email, created_at)`. Эта команда добавит копии email и created_at прямо в листовые узлы индекса. Теперь, когда PostgreSQL найдёт нужную запись в индексе, он увидит, что все необходимые ему данные уже здесь и ему не нужно делать лишний «прыжок» в таблицу. Такой запрос выполняется по плану Index Only Scan и работает максимально быстро.

Предположим, что ты всегда ищешь пользователей по email без учёта регистра, то есть твой запрос выглядит как `WHERE LOWER(email) = ....` Обычный индекс на email здесь не поможет, потому что ты ищешь не по самому значению, а по результату функции LOWER(). Но ты можешь создать индекс прямо по этому выражению: `CREATE INDEX ON users (LOWER(email))`. Теперь PostgreSQL сможет использовать его для таких запросов. Это работает для любых неизменяемых (immutable) функций.

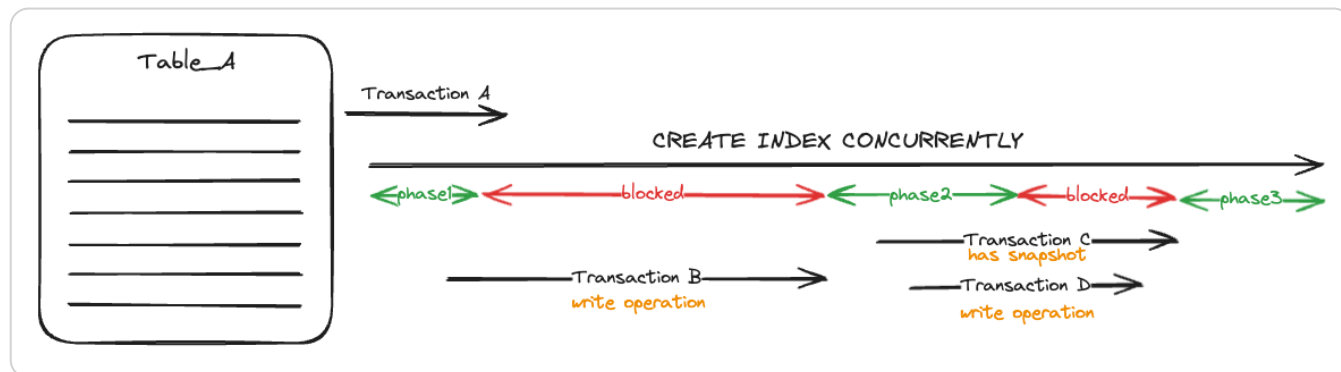


А теперь разберём одну из самых частых и коварных проблем производительности в веб-приложениях — страничную навигацию. Классический способ сделать «пейджинг» — использовать LIMIT и OFFSET. На первой странице OFFSET 0, на второй OFFSET 10 и так далее. Это прекрасно работает на первых 10–20 страницах. Но OFFSET — это ловушка. Когда ты просишь OFFSET 10000, ты не говоришь базе «прыгни сразу к 10001-й записи». Ты даёшь ей указание: «Найди все записи, отсортируй их, отсчитай первые 10000, выброси их, а потом отдай мне следующие 10». С каждой новой страницей база делает всё больше и больше бесполезной работы. Это «убивает» производительность на больших наборах данных. Более того, страницы нестабильны: пока ты смотришь, новые данные могут быть добавлены в начало, и вся нумерация «поедет».

Решение этой проблемы — изменить саму парадигму. Вместо того чтобы считать от начала, мы будем двигаться от курсора. Этот подход называется Keyset Pagination, или пагинация по набору ключей. Идея проста: когда мы загружаем страницу, мы запоминаем значения сортируемых полей у последней записи. Эти значения и есть наш курсор. Для загрузки следующей страницы мы просим базу: «Дай мне 10 записей, у которых дата регистрации меньше, чем у моего курсора». Посмотри на WHERE-условие. Мы используем сравнение кортежей (tuple comparison), что очень эффективно. Обрати внимание, что мы сортируем и фильтруем по паре (reg_date, id). Добавление id критически важно. Это «тай-брейкер». Если два пользователя зарегистрировались в одну и ту же микросекунду, уникальный id гарантирует, что у нас всегда будет строгий и стабильный порядок, и мы никого не пропустим и не покажем дважды.

Почему это работает быстро? Потому что WHERE условие в нашем новом запросе идеально использует составной B-Tree-индекс. Для базы данных такая задача — это не «отсчитать 10000», а «найти в отсортированном списке закладку и прочитать 10 записей после неё». Это чрезвычайно быстрая операция Index Scan. В результате время ответа на запрос 10-й страницы и 10000-й страницы будет практически одинаковым. Однако у этого подхода есть компромиссы. Самый главный — мы теряем возможность прыгать на произвольную страницу. Мы можем двигаться только последовательно. Именно поэтому в социальных сетях ты видишь кнопку «Загрузить ещё», а не список страниц. Логика на клиенте и сервере немного усложняется, так как нужно передавать этот курсор, но выигрыш в производительности и стабильности данных огромен.

Мы говорили, что PostgreSQL индексирует NULL. Но что, если они нам не нужны? В таблице users есть поле deleted_at, которое NULL для 99% «живых» пользователей. Мы никогда не ищем по NULL, только по конкретным датам. Мы можем создать очень эффективный частичный индекс: CREATE INDEX ... WHERE deleted_at IS NOT NULL;. Он будет крошечным, так как будет содержать только удалённых пользователей и не будет замедлять UPDATE для всех остальных.



Очень важный момент для работающих систем. Обычная команда `CREATE INDEX` на большой таблице блокирует её для записи. Твой сайт или сервис может «встать» на минуты или даже часы. Это недопустимо. Для этого существует команда `CREATE INDEX CONCURRENTLY`. Она строит индекс в фоновом режиме, не мешая работе пользователей. Это дольше, создаёт больше нагрузки, но не вызывает простоя. Единственное правило — её нельзя выполнять внутри транзакции. `CREATE INDEX CONCURRENTLY` не просто работает «в фоне». Он выполняется в три фазы сканирования и зависит от блокировок `VirtualXID`. На второй и третьей фазе процесс обязан дожидаться завершения `VCEX` транзакций, которые начались до него, даже тех, которые просто читают данные (`SELECT`) из других таблиц. Если в твоей системе есть долгая аналитическая транзакция или «зависшая» сессия (`Idle in transaction`), миграция с `CIC` остановится и заблокирует изменение схемы таблицы на часы, что может привести к исчерпанию пула соединений.

Почему `CREATE INDEX CONCURRENTLY` может «повесить» Production?

- **Сценарий:** ты запускаешь создание индекса в 12:00. В это же время в другом соединении открыта транзакция (например, отъём), которая просто «висит» и ничего не делает. `CIC` обязан дожидаться окончания всех транзакций, которые могли бы изменить данные в таблице. Пока отчёт не закроется, создание индекса не перейдёт к фазе 2. Пока `CIC` висит в ожидании, он может удерживать защёлки на системных каталогах. Другие операции `ALTER TABLE` или `VACUUM` встанут в очередь за ним. Начинается «запирание» всей базы.
- **Правило:** никогда не запускай `CIC`, не проверив `pg_stat_activity` на наличие «забытых» долгих транзакций.

Подведём итог всему, что мы сегодня узнали о типах индексов. Перед тобой краткое резюме.

- Если у тебя классическая задача — найти по ID, отфильтровать по дате, отсортировать по имени — в 99% случаев твой выбор — это B-Tree. Это твой инструмент по умолчанию.
- Если тебе нужно «заглянуть внутрь» сложного поля, будь то массив тегов или JSON-документ, и найти там определённый элемент, твой выбор — GIN.
- Если ты работаешь с геоданными, ищешь пересечения или ближайших «соседей», тебе нужен GiST.
- И если перед тобой гигантская, физически отсортированная по какому-то полю таблица логов и ты хочешь сэкономить десятки гигабайт мест, посмотри в сторону BRIN.
- Правильный выбор типа индекса — это первый шаг к эффективному запросу.

И в заключение я хочу дать несколько золотых правил, которыми стоит руководствоваться в твоей работе.

- **Первое и главное:** анализируй, потом создавай. Твой лучший друг — это `EXPLAIN ANALYZE`. Не добавляй индексы, пока не докажешь с его помощью, что они действительно нужны и что планировщик их использует.
- **Второе:** Индексируй с умом. Индекс на поле «пол» (`gender`), где всего два значения, почти бесполезен. А вот индекс на `email` или `username` очень эффективен.
- **Третье:** порядок имеет значение. Тщательно продумывай порядок столбцов в составных индексах, чтобы один индекс мог обслуживать как можно больше твоих запросов.
- **Четвёртое:** думай о покрытии. Если ты часто запрашиваешь 2–3 поля, подумай, не стоит ли создать покрывающий индекс, чтобы избежать лишних чтений из основной таблицы.

- **Пятое:** используй продвинутые техники. Частичные и функциональные индексы могут показаться сложными, но иногда именно они дают максимальный прирост производительности.
- **И шестое:** убирай за собой. Ненужные индексы — это технический долг. Они замедляют систему. Регулярно проводи аудит и избавляйся от того, что не используется.